

Programare Orientată Pe Obiecte

Curs 2

Tipuri de date, structuri de control și operatori în Java
Clase și Obiecte: constructori și metode

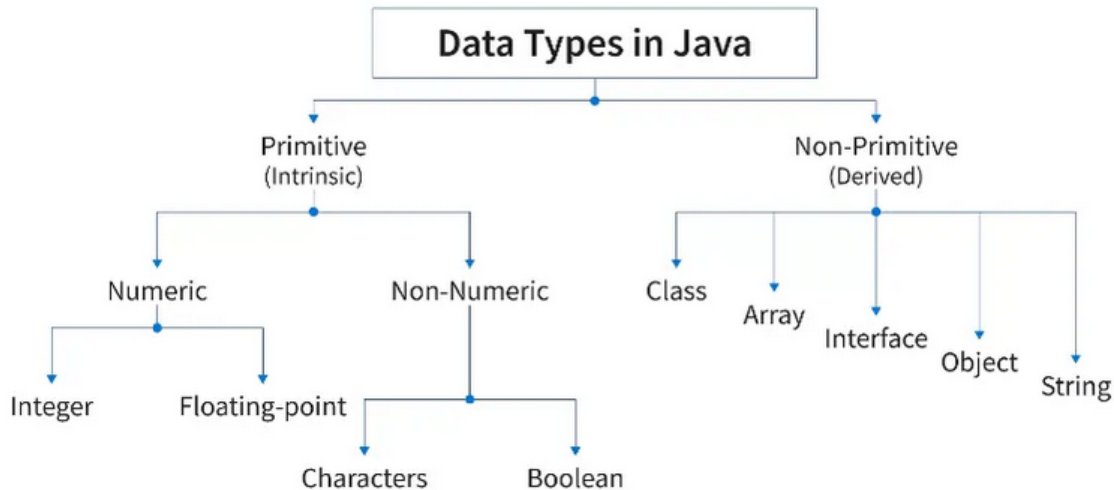
Octombrie 2025

Agenda

- 1 Tipuri de date în Java
- 2 Operatori și structuri de control
- 3 Constante, comentarii, documentație în JAVA
- 4 Clase și Obiecte
- 5 Quizz - fixare cunoștințe

Tipuri de date în Java

Tipuri de date în JAVA



Tipuri de date primitive în JAVA

TYPE	DESCRIPTION	DEFAULT	SIZE	EXAMPLE LITERALS	RANGE OF VALUES
boolean	true or false	false	1 bit	true, false	true, false
byte	twos complement integer	0	8 bits	(none)	-128 to 127
char	unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\l', '\', '\n', ' β'	character representation of ASCII values 0 to 255
short	twos complement integer	0	16 bits	(none)	-32,768 to 32,767
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2	-2,147,483,648 to 2,147,483,647
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F	upto 7 decimal digits

Tipuri de date primitive și clase învelitoare

- Clasele învelitoare sunt clasele care încapsulează tipurile primitive, astfel încât să se poată manipula ca niște obiecte și pentru a le putea 'atașa' metode (în special cu rol de conversie)
- Exista o clasa wrapper pentru fiecare tip primitiv !

Tipul primitiv	Clasa Învelitoare	Argumentele constructorilor
byte	Byte	byte sau String
short	Short	short sau String
int	Integer BigInteger	int sau String String
long	Long	long sau String
float	Float	float, double sau String
double	Double BigDecimal	double sau String String
char	Character	char
boolean	Boolean	boolean sau String

Clase învelitoare

Boxing și unboxing

- Boxing (împachetare): procesul de trecere de la o valoare primitivă la un obiect al clasei sale învelitoare.

```
int i = 288;  
Integer iw = new Integer(i);
```

- Unboxing (despachetare): procesul de trecere de la un obiect al unei clase învelitoare la valoarea corespunzătoare a unui tip primitiv

```
int uw = iw.intValue();
```

```
import java.util.ArrayList;  
public class DemoBoxing {  
    public static void main(String[] args) {  
        Integer a = 10;  
        System.out.println(a);  
        ArrayList<Integer> lst = new ArrayList<  
            Integer>();  
        lst.add(20);  
        lst.add(a);  
        System.out.println(lst);  
        int b = lst.get(0);  
        System.out.println(b);  
    }  
}
```

Clase învelitoare

Boxing și Unboxing automat

```
class WrapperDemo{
    WrapperDemo() {}
    void takeNumber(Integer i){System.out.println("i este " + i);}
    int giveNumber(){ Integer a = 20; return a; }
}

public class WrapperExamples{
    public static void main(String args[]){
        WrapperDemo w = new WrapperDemo();
        //argumente pentru metode
        w.takeNumber(12);
        //valori returnate
        int x = w.giveNumber();
        System.out.println("x " + x);
        Boolean b = true;
        if (b) {System.out.println("true");} //Expresii boolean
        Integer a = 23;
        a++; // Operatii cu numere
        System.out.println("a este " + a);
        Double j = 1098.2;
        Double k = j + 2; //Atribuirii
        System.out.println("k este "+ k);
    }
}
```

Verificarea egalității și constantele în clasele învelitoare

Pentru a verifica dacă două variabile de tip referință au aceeași valoare, vom folosi metoda `equals()`:

```
Integer i1 = 42; // împachetare automata
Integer i2 = 42; // împachetare automata
if (i1 != i2) // nu este garantat ca sunt diferite
    System.out.println("obiecte diferite");
if (i1.equals(i2)) // garantat ca sunt identice!
    System.out.println("Variabile cu valori egale");
```

Constante pentru valori maxime și minime pentru tipurile primitive:

```
Integer.MAX_VALUE, Integer.MIN_VALUE, Double.MAX_VALUE, Double.MIN_VALUE, etc.
```

Clasa `Boolean` are nume pentru două constante de tipul `Boolean`: `Boolean.TRUE` și `Boolean.FALSE` sunt obiectele booleene corespunzătoare valorilor `true` și `false` ale tipului primitiv `boolean`.

Clasa String

- Nu există un tip primitiv pentru șiruri în Java; Clasa String este o clasă Java predefinită folosită la stocarea și prelucrarea șirurilor de caractere
- Obiectele de tipul String sunt compuse din șiruri de caractere scrise între ghilimele; Orice șir scris între ghilimele este de clasă String: *"Curs de POO"*
- Metode de creare a unui String în Java:

```
String s1 = "CursJAVA";  
String s2 = new String ("Curs");
```

- Concatenarea stringurilor se realizează folosind operatorul +

```
String s1 = "Buna ziua, ";  
String s2 = "colegi";  
String s3 = s1 + s2 // "Buna ziua, colegi"
```

- Orice număr de șiruri pot fi concatenate;
- La combinarea unui șir de caractere cu aproape orice alt tip, rezultatul este un șir de caractere

```
String s3 = "valoarea este " + 42
```

Metodele clasei String

- O metodă a clasei String se apelează prin numele unui obiect String, un punct, numele metodei și o pereche de paranteze între care sunt cuprinse argumentele (dacă sunt)
- Poziția sau indexul unui caracter într-un șir se numără întotdeauna de la zero
- Lungimea șirului *s*: `int i = s.length()`

Clasa String

Metode pentru a compara String-uri

Folosiți aceste metode în loc de `==` și `!=`

- `i = s.compareTo(t)`: compară `s` cu `t`. Returnează < 0 dacă $s < t$, 0 dacă `==`, > 0 dacă $s > t$
- `i = s.compareToIgnoreCase(t)`: compară `s` cu `t` fără a ține cont de majuscule / minuscule. Returnează < 0 dacă $s < t$, 0 dacă `==`, > 0 dacă $s > t$
- `b = s.equals(t)`; Returnează `true` dacă `s` și `t` au valori egale
- `b = s.equalsIgnoreCase(t)`; compara fara a tine cont de minuscule / majuscule
- `b = s.startsWith(t)`; returnează `true` dacă `s` începe cu `t`
- `b = s.startsWith(t, i)`; returnează `true` dacă `t` apare începând cu indexul `i`
- `i = s.indexOf(c)`; returnează indicele caracterului `c`

Clasa String

Metode ale clasei String

Pentru a obține substringuri (părți dintr-un String):

- `c = s.charAt(i)`: caracterul de la poziția `i` din `s`.
- `s1 = s.substring(i)`: subșirul de la indexul `i` până la sfârșitul lui `s`.
- `s1 = s.substring(i, j)`: subșirul de la indexul `i` până înainte de indexul `j` din `s`.

Crearea unui nou șir din original:

- `s1 = s.toLowerCase()`
- `s1 = s.toUpperCase()`
- `s1 = s.trim()`
- `s1 = s.replace(c1, c2)`

Metode statice pentru conversia la String:

- `s = String.valueOf(x)`
- `s = String.format(f, x...)`

Seturi de caractere

- ASCII: Set de caractere folosit de multe limbaje de programare, care conține toate caracterele folosite în mod normal pe o tastatură pentru limba engleză plus câteva caractere speciale; Fiecare caracter este reprezentat de un cod numeric
- Unicode: Set de caractere folosit de limbajul Java care include tot setul ASCII plus multe dintre caracterele folosite cu alfabetul elatin

Exemple:

```
char c='\u0103'; // litera 'ă'
String s="\u00eencoto\u015fm\u0103ni\u0163i";
//încotoșmăniți
```

Tipuri de date referință

- Toate obiectele sunt create în memorie (pe heap – zona de memorie unde se face alocarea dinamică)
- Pentru a accesa datele dintr-un obiect sau a lucra cu un obiect, este nevoie de o variabilă pe stivă, variabilă care poate stoca o referință la adresa obiectului
- Despre variabilele care stochează referințe la adrese de obiecte se spune că păstrează tipul de date referință
- Exemplu: `Dog t = new Dog("Patrocle");`
- în Java nu exista variabile obiecte, exista variabile referință la obiecte
- O variabila referinta stocheaza biti care reprezinta modalitatea de a accesa un obiect.
- Nu stocheaza obiectul in sine, ci stocheaza o adresa.

Variabilele referință (obiecte)

Declarare, creare și asignare

Cei trei pași de declarare, creare și asignare pentru un obiect:

1 Dog myDog **3** = **2** new Dog ();

1. Declara o variabila referinta:

Dog myDog = new Dog();



2. Creaza un obiect

Dog myDog = new Dog();



3. Leaga obiectul si referinta

Dog myDog = new Dog();



- JVM alocă spațiu pentru o variabilă referință.
- Numeste acea variabilă myDog.
- Variabila referință myDog este pentru totdeauna de tipul Dog (controlează un Dog nu Cat sau alt obiect).

- JVM alocă spațiu pentru un nou obiect Dog, pe heap.

- Atribue noul obiect Dog la variabila referință myDog.

Compararea obiectelor

La compararea a două variabile, se compară conținutul lor;

- În cazul obiectelor, conținutul este adresa unde este stocat obiectul (adică se vor compara referințele)
 - Șirurile de caractere în Java sunt obiecte ale clasei String
 - Clasa String furnizează metode de comparare
- Cea mai bună metodă în ceea ce privește compararea obiectelor este să definim metode de comparare pentru clasa respectivă
- Atenție la diferența dintre metoda equals() și operatorul ==
 - Equals() testează egalitatea de conținut
 - == testează egalitatea adreselor (dacă ambele obiecte refera aceeași zonă de memorie)

TODO EXEMPLU

Exercițiu

Formular de Înregistrare cu Validare în Java

Dezvoltați o aplicație JAVA pentru crearea unui formular de înregistrare simplu. Formularul colectează date de la utilizator și validează intrările conform regulilor.

- Câmpuri: Nume Complet (text); Email (text); Telefon (text); Parolă (Input parolă); Confirmare Parolă (Input parolă); Vârstă (numeric)
- Validare câmpuri:
 - Toate câmpurile trebuie să aibă conținut
 - Nume Complet: conține cel puțin 3 caractere; conține doar litere și spații.
 - Email: format valid de email și nu conține caractere invalide.
 - Telefon: conține exact 10 cifre; Nu are litere sau simboluri speciale.
 - Parolă: lungime de cel puțin 8 caractere; Conține cel puțin: O literă majusculă, o literă minusculă, o cifră, un caracter special (de ex., @, #, \$, %, !, etc.).
 - Confirmare Parolă: corespunde cu câmpul Parolă.
 - Vârstă: Trebuie să fie un număr întreg valid între 18 și 100.

Exercitiu

exemplu metode

```
import java.util.Scanner;
...
public static Boolean containsOnlyDigits(String str){
    for (char c : str.toCharArray()) {
        System.out.print(c);
        if (!Character.isDigit(c)) return false;
    }
    return true;
}
public static void main(String[] args) {
    String nume;
    //citire de la tastatura - user input
    Scanner reader = new Scanner(System.in);
    System.out.println("Dati numele: ");
    nume = reader.nextLine();
    ...
}
```

Exercitiu

Validare adresa email: `java.util.regex.*`

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
class Demo {
    public static boolean validEmail(String email) {
        String emailRegex = "[a-zA-Z0-9_+&*-]+(?:\\.|"+
            "[a-zA-Z0-9_+&*-]+)*@" +
            "(?:[a-zA-Z0-9-]+\\.)+[a-zA-Z]{2,7}$";

        Pattern pat = Pattern.compile(emailRegex);
        if (email == null) return false;
        return pat.matcher(email).matches();
    }

    public static void main(String[] args) {
        String address = "exemplu@gmail.com";
        if (validEmail(address)) System.out.println("Valid!");
        else System.out.println("Email Invalid!");
    }
}
```

Operatori și structuri de control

Operatori în JAVA

Vedeți lucrarea de laborator

Câteva diferențe față de C:

- Concatenarea pentru String: operatorul +
- Operatorul pe biți $\ggg$; D.e. $n \ggg p$; deplasează biții lui n spre dreapta cu p poziții. În pozițiile de rang superior se inserează zerouri.

```
System.out.println(Integer.toBinaryString(-2));  
//afiseaza 1111111111111111111111111111110 (32 biti)  
System.out.println(Integer.toBinaryString(-2>>1)); //afiseaza ?  
System.out.println(Integer.toBinaryString(-2>>>1)); // afiseaza  
System.out.println(Integer.toBinaryString(-2>>>16)); //afiseaza ?
```

- Operatori pentru lucrul cu obiecte – vor fi tratați în detaliu mai târziu în detaliu mai târziu

Precedența operatorilor

Precedența operatorilor	
<code>. [] (args) post ++ --</code> <code>! ~ unary + - pre ++ --</code> <code>(type) new</code> <code>* / %</code> <code>+ -</code> <code><< >> >>></code> <code>< <= > >= instanceof</code> <code>== !=</code> <code>&</code> <code>^</code> <code> </code> <code>&&</code> <code> </code> <code>? :</code> <code>= += -= etc</code>	Tineți minte precedența pentru operatorii unari, <code>* / %</code> <code>+ -</code> <i>comparații</i> <code>&& </code> <i>= asignări</i> Folosiți paranteze (<code>()</code>) pentru ceilalți

Instrucțiunea if

Instrucțiunea if specifică ce bloc de cod să se execute în funcție de rezultatul evaluării unei condiții numită expresie booleană.

```
if (<expresie booleană>)  
<then> Instr1  
else  
<else> Instr 2
```

Sau:

```
(<expresie booleană>)? Instr1 : Instr2  
ex: int max = (a>b)? a:b;
```

- Sintaxă similară limbajului C, dar atenție la ce este o expresie booleană în Java

Sugestii pentru if

- Începeți testul cu cazul cel mai relevant
- Face codul mai ușor de citit
- Nu uitați de clauza else!
- Evitați condițiile complicate
- Divizați condițiile în variabile/funcții booleene
- Încercați să folosiți condiții pozitive
- Exemplu – se preferă a doua variantă

```
//varianta 1
if (!node.isFirst() && node.value() != null)
    instr1
else
    instr2
//varianta 2
if (node.isFirst() || node.value() == null)
    instr2
else
    instr1
```

Instrucțiunea switch

Sintaxa și exemplu

```
switch(expresie) {  
    case x:      // code block  
        break;  
    case y:      // code block  
        break;  
    default:     // code block  
}
```

Tipul de dată al `< expresie >`s trebuie să fie char, byte, short, int

```
int month = 8; String monthString;  
switch (month) {  
    case 1: monthString = "January";  
        break;  
    case 2: monthString = "February";  
        break;  
    // . . .  
    case 12: monthString = "December";  
        break;  
    default: monthString = "Invalid month";  
        break;  
}  
System.out.println(monthString);
```

Instrucțiunea switch

- Dacă există o valoare care se potrivește cu valoarea expresiei, atunci se execută corpul acesteia. Altfel execuția continuă cu instrucțiunea care urmează instrucțiunii switch
- Instrucțiunea break face să nu se execute porțiunea care urmează din switch, ci să se continue cu ceea ce vine după switch
- break este necesar pentru a executa instrucțiunile dintr-un caz, și numai unul (iarăși, ca în C)
- Sugestii pentru switch:
 - Ordonăți cazurile (logic sau alfabetic)
 - Prevedeți întotdeauna cazul implicit (default)
 - Întotdeauna folosiți break între cazuri
 - Încercați să păstrați mică dimensiunea instrucțiunii switch
 - Divizați cazurile de mari dimensiuni în funcții

Instrucțiuni repetitive

while, do-while, for

- Instrucțiunile repetitive controlează un bloc de cod de executat un număr fixat de ori sau până la îndeplinirea unei anumite condiții
- Ca și C, Java are trei instrucțiuni repetitive:

```
while (condition) {  
    // code block to be  
    executed  
}
```

```
do {  
    // code block to be  
    executed  
}  
while (condition);
```

```
for (statement1; statement2; statement3) {  
    // code block to be executed  
}
```

- Instrucțiunea while este o buclă cu pre-testare (testul se face la intrarea în buclă): Corpul buclei poate să nu fie executat niciodată
- Instrucțiunea do-while este o buclă cu post-testare: Corpul ciclului este executat cel puțin o dată

Exemplu: while, do-while

```
int countdown = 3;
while (countdown > 0) {
    System.out.println(countdown);
    countdown--;
}
System.out.println("Happy Year!!");
```

```
int i = 0;
do {
    System.out.println(i);
    i++;
}
while (i < 5);
```

La instrucțiunile repetitive este important să se asigure terminarea ciclului la un moment dat. Exemple de bucle infinite:

```
int item = 0;
while (item < 5000) {
    item = item * 5;
}
```

```
int count = 1;
while (count != 10) {
    count = count + 2;
}
```

Numerele reale nu se folosesc în teste exacte sau incrementări, deoarece valorile lor sunt reprezentate cu aproximație.

Exemplu: for

```
int sum = 0;
for (int i = 1; i <=100; i++){
    sum += i;
}
```

- Variabila *i* din exemplu se numește variabilă de control (contorizează numărul de repetiții)
- *< increment >* poate fi orice cantitate
- Din nou, ca în C

Buclele for sunt ideale atunci când numărul de iterații este cunoscut

- Folosiți o instrucțiune pentru fiecare parte
- Declarați variabilele de ciclu în antet (reduce vizibilitatea și interferența)
- Nu se recomandă modificarea variabilei de control în corpul ciclului

Exemplu:

for pentru iterații peste colecții și tablouri

```
public class ForEachDemo {  
    public static void main(String[] args) {  
        int[] a = {32, 87, 3, 589, 12, 1076};  
        for (int el : a) {  
            System.out.print(element + " ");  
        }  
    }  
}
```

Afișare la ieșirea standard:

32 87 3 589 12 1076

- Creat special pentru iterații peste colecții și tablouri
- Nu funcționează oriunde (d.e. nu se pot accesa indicii de tablouri)

Constante, comentarii, documentație în JAVA

Numirea constantelor

- În loc de numere "anonime", declarați constante simbolice (cu nume) și folosiți-le numele:

```
public static final double CM_PER_INCH = 2.54;  
public static final int HOURS_PER_DAY = 24;
```

- Previne schimbarea nedorită a valorii
- Ușurează modificarea valorii
- Convenția de nume pentru constante
 - Toate literele majuscule
 - Limitele de cuvinte marcate prin liniuța de subliniere

Comentarii și documentarea programului

Comentarii:

- Comentariu de o linie: începe cu simbolurile `//` și provoacă ignorarea a ceea ce urmează până la sfârșitul liniei
- Comentariul bloc: La fel ca în C (perechea `/*, */`); Furnizează documentație utilizatorilor programului

Documentarea programului:

- Java include programul `javadoc` care extrage automat documentația din comentariile bloc din clasele definite, dacă începutul comentariului are un asterisc suplimentar (`/**`)
- Un program bine scris este autodocumentat: structura sa se clarifică prin alegerea numelor de identificatori și modelul de indentare

Clase și Obiecte

Anatomia unei clase

```
public class Taxi
{ // VARIABILE INSTANTA
  private String driver;
  private int km;

  //CONSTRUCTOR:
  //construct a Taxi copying in the data into the instance variables
  public Taxi(String initDriver, int initKm){
    driver = initDriver;
    km = initKm;
  }
  // METODE
  public void print() {
    System.out.println("Taxi Driver: " + driver);
    System.out.println("Total Killometers: " + km);
  }
  public int getKm(){
    return km;
  }
  public void drive(int kms) {
    this.km += kms;
  }
  ...
}
```

Constructorii

Scop: inițializează starea unui obiect

Inițializează toate atributele obiectului. Dacă acestea nu se inițializează explicit, ele vor fi inițializate implicit cu valorile inițiale din Java, în funcție de tipul lor.

- **Nume:** La fel cu numele clasei. Prima literă mare.
- **Intrare:** 0 sau mai mulți parametri.
- **Ieșire:** Nu este tip de retur în antet.
- **Numar de apelari:** se apelează cel mult o dată pe obiect; este invocat de operatorul "new".

Cod:

```
public Taxi() ...
```

Constructori multipli

```
public class Taxi
{
    // VARIABLE INSTANTA
    private String driver;
    private int km;

    //CONSTRUCTORI MULTIPLI:
    public Taxi(){
        km = 0;
        driver = "Unknown";
    }
    public Taxi(String initDriver, int initKm){
        driver = initDriver;
        km = initKm;
    }
    public static void main(String[] args){
        Taxi t = new Taxi();
    }
}
```

O operație "new" încununată de succes creează un obiect pe heap și execută constructorul al cărui listă de parametri "corespunde" listei sale de argumente (ca număr, tip, ordine).

Folosirea corespunzătoare a constructorilor

Un constructor ar trebui întotdeauna să creeze obiectele într-o stare validă:

- Constructorii nu trebuie să facă nimic altceva decât să creeze obiecte
- Dacă un constructor nu poate garanta că obiectul construit este valid, atunci ar trebui să fie `private` și accesat prin intermediul unei metode de fabricare (factory method)
- Notă: în general, termenul de metodă de fabricare este folosit pentru a se referi la orice metodă al cărei scop principal este crearea de obiecte

Factory method

Factory method: metodă statică care apelează un constructor (Constructorul este de obicei private)

- Metoda de fabricare poate determina dacă să invoce sau nu constructorul
- Metoda de fabricare poate arunca o excepție, sau poate face altceva potrivit în cazul în care i se dau argumente ilegale sau nu poate crea un obiect valid

```
public static Taxi create(int kms) {  
    if (kms < 0)  
        throw new IllegalArgumentException("Too few kms!");  
    else  
        return new Taxi(kms);  
}
```

Metode

Scop:

Execută comportamentul obiectului

- **Nume:** Un verb; Începe cu literă mică.
- **Intrare:** 0 sau mai mulți parametri.
- **Ieșire:** Pentru ieșire este nevoie de un tip returnat.
- **Numar de apelari:** nelimitat pentru un obiect; se apelează folosind operatorul "punct":
`numeObiect.numeMetoda(argumente)`

Cod:

```
public void turnLeft() ...
```

Ce poate face o metodă:

- Schimbă starea obiectului său
- Raportează starea obiectului său
- Operează asupra numerelor, a textului, a fișierelor, graficii, paginilor web, hardware, etc.
- Poate crea alte obiecte
- Poate apela o metodă a unui alt obiect: `obiect.metoda(args)`
- Poate apela o metodă din aceeași clasă: `this.metoda(args);`
- Se poate autoapela (recursivitate)

Intrarea pentru o metodă

- O metodă poate primi 0 sau mai multe intrări
- Intrările pe care le așteaptă o metodă sunt specificate prin lista de "parametri formali" (tip nume1, tip nume2, ...)
- La apelul unei metode, numărul, ordinea și tipul argumentelor trebuie să se potrivească cu parametrii corespunzători

Declararea metodei (cu parametri)	Apelul metodei (cu argumente)
<code>public void meth1 () { .. }</code>	<code>obj.meth1 ()</code>
<code>public int meth2 (boolean b) { .. }</code>	<code>obj.meth2 (true)</code>
<code>public int meth3 (int x, int y, Taxi t) { .. }</code>	<code>obj.meth3 (3, 4, cab)</code>

Ieșirea unei metode

- O metodă trebuie să returneze ceva: void sau non-void
- Dacă nu returnează nimic (nu are ieșire), tipul returnat este "void"
public void setKm(int km)..
- Dacă tipul returnat este non-void (d.e. int, boolean, Taxi) atunci metoda trebuie să conțină o instrucțiune return cu o valoare; valoarea returnată trebuie să se potrivească cu tipul returnat.

```
public int getKm() {  
    return km;  
}
```

Metode setter și getter

mutatoare și accesoare

```
public class Taxi{
    private int km;
    public Taxi(){
        km = 0;
    }
    // raporteaza # km
    public int getKm(){
        return km;
    }
    // seteaza (schimba) # km
    public void setKm(int m){
        km = m;
    }
    public static void main(String[] args){
        Taxi cabl;
        cabl = new Taxi();
        cabl.getKm(); // 0
        cabl.setKm(500);
        cabl.getKm(); //500
    }
}
```

Exercițiu: constructori și metode

Scrieți o aplicație Java care să permită gestionarea și controlul informațiilor despre taxiuri.

- Pe măsură ce avansăm în conceptele de programare orientată pe obiecte, vom actualiza cerințele și vom crea o aplicație care simulează un sistem cababil să urmărească taxiurile, șoferii și cursele clienților, oferind funcționalități de bază pentru adăugarea, actualizarea și vizualizarea înregistrărilor.
- Deocamdată, utilizând cunoștințele acumulate până la cursul de azi vom crea clasele de mai jos cu constructori și metode relevante:
 - Taxi
 - Șofer
 - Client
 - Locație / Adresă

Exercițiu: constructori și metode

Clase:

- Clasa Taxi:
 - Câmpuri: taxiID, model, numărDeÎnmatriculare, status (disponibil, înCursă, înMentenanță)
 - Metode: getters/setters, displayTaxiDetails(), updateStatus()
- Clasa Șofer:
 - Câmpuri: driverID, nume, numărPermis, taxiAtribuit
 - Metode: getters/setters, displayDriverDetails()
- Clasa Client:
 - Câmpuri: clientID, nume,
 - Metode: getters/setters, displayClientDetails()
- Clasa Adresa:
 - Câmpuri: strada, numar, oras, tara
 - Metode: getters/setters, displayAddressDetails()

Quizz - fixare cunoștințe

Q1/6 — Identifică constructorii

Care dintre semnăturile de mai jos sunt **constructori** ai clasei `Person`? (alege toate variantele corecte)

```
class Person {  
    private String nume;  
    private int varsta;  
  
    public Person() {  
        this("Anonim", 0);  
    }  
  
    public Person(String nume, int varsta) {  
        this.nume = nume;  
        this.varsta = varsta;  
    }  
  
    public void Person(String x) { // <- atentie  
        this.nume = x;  
    }  
  
    public String getNume() {  
        return nume;  
    }  
}
```

- A. `Person()`
- B. `Person(String, int)`
- C. `void Person(String)`
- D. `getNume()`

Q2/6 — Constructorul setează corect câmpul?

În clasa de mai jos, constructorul setează corect câmpul `model`? (DA/NU)

```
class Car {  
    private String model;  
  
    public Car(String model) {  
        model = model; // umbrire parametrul-camp  
    }  
}
```

Dacă răspunsul tău este **NU**, scrie pe scurt corecția în stil Java.

Q3/6 — Setter corect?

Este următorul setter corect ?

```
class Car {  
    private String model;  
  
    public boolean setModel(String model) {  
        this.model = model;  
        return true;  
    }  
}
```

- A. Da, orice tip de return e acceptat
- B. Nu, un setter JavaBeans ar trebui să fie `void`
- C. Ar trebui să returneze `this` (fluent API) în loc de `boolean`

Q4/6 — Getter corect?

Care dintre variantele de getter este corectă pentru model?

```
class Car {  
    private String model;  
  
    // (I)  
    public String getModel() {  
        return this.model;  
    }  
  
    // (II)  
    public String getModel() {  
        return model = "Default";  
    }  
}
```

- A. Doar (I)
- B. Doar (II)
- C. Ambele sunt ok

Q5/6 — Supraincărcare de constructori

Câți **constructori** are clasa Book? (alege varianta corectă)

```
class Book {  
    private String title;  
    private int pages;  
  
    Book() { // (1)  
        this("Unknown", 0);  
    }  
  
    public Book(String title) { // (2)  
        this.title = title;  
    }  
  
    public void Book(int pages) { // (3) <-- e metoda, nu constructor  
        this.pages = pages;  
    }  
}
```

- A. 1 constructor
- B. 2 constructori
- C. 3 constructori

Q6/6 — Identifică erorile (constructor vs metodă)

Ce **nu** este corect în clasa Student? (alege toate variantele)

```
class Student {  
    private String name;  
    private int year;  
  
    public Student() {  
        this("Anonim", 1);  
    }  
    public Student(String name, int year) {  
        this.name = name;  
        this.year = year;  
    }  
    public int Student(String name) {  
        this.name = name;  
        return 1;  
    }  
    void setYear(int y) {  
        if (y > 0) this.year = y;  
    }  
    int getYear() {  
        return year;  
    }  
}
```

- A. constructori
- B. setteri
- C. getteri

Răspunsuri

- ❶ **Q1: A, B.** *Person()* și *Person(String,int)* sunt constructori (numele = numele clasei, **fără** tip de return). *void Person(String)* este metodă, nu constructor. *getNum()* este metodă.
- ❷ **Q2: NU.** Linia `model = model;` folosește parametrul peste câmp (umbrire). Corect:

```
public Car(String model) { this.model = model; }  
// Stil JavaBeans:  
public void setModel(String model) { this.model = model; }
```
- ❸ **Q3: B.** Un setter JavaBeans ar trebui să fie `public void setX(T value)` (fără return). Varianta dată întoarce `boolean`, deci nu respectă convenția.
- ❹ **Q4: A.** (I) este un getter corect. (II) modifică starea (`model = "Default"`) în getter, ceea ce e greșit; în plus ar reseta valoarea la fiecare apel.
- ❺ **Q5: B** (2 constructori). *Book()* și *Book(String)* sunt constructori. *void Book(int)* are tip de return deci este metodă, nu constructor.
- ❻ **Q6: A, B, C.** (c) nu e constructor (are tip de return `int`); (d) nu respectă convenția setterului (ar trebui `public void setYear(int)`); (e) nu respectă convenția getterului (ar trebui `public int getYear()`).